provide me in three responses

line 1-150
line 151 - 272
line 272 - end of the file (too much detailed explanation is not required)

You asked for three responses, with the first covering roughly “lines 1–150”. The YAML doesn’t literally show line numbers here, but I’ll treat this first response as:

- From the top of the file
- Down through:
  - Workflow header and trigger
  - Job/step setup
  - All helper functions (`parseBranch`, `compareVersionOnly`, `sleep`, `getPullRequest`, `pollMergeability`, `addLabel`, `addComment`, `mergePullRequest`)
  - Parsing the current branch and building `eligibleBranches`

I’ll keep explanations precise but not over-verbose, and use small examples. Then next responses will cover the later parts.

1. Workflow header and trigger

```
name: Cascade Next PR

on:
  pull_request_target:
    types: [closed]

concurrency:
  group: cascade-next-${{ github.event.pull_request.number || github.run_id }}
  cancel-in-progress: true

permissions:
  contents: read
  pull-requests: write
  issues: write
```

- `name: Cascade Next PR`
  Label for this workflow in the Actions UI.
- `on: pull_request_target: types: [closed]`
  Run this workflow when a PR in the repo is closed.

Combined with a later condition, it will actually act only when the PR is closed **and** merged.

- `concurrency.group: cascade-next-${{ github.event.pull_request.number || github.run_id }}`
  Ensures only one run per PR at a time.
  For PR #42, the group is `cascade-next-42`; if a new run starts for the same PR, the previous run is cancelled.
- `permissions:`
  - `contents: read`: can read repo code.
  - `pull-requests: write`: can create, label, comment on, and merge PRs.
  - `issues: write`: can add labels/comments (PRs are issues in the API).[1]

2. Job and `github-script` step

```
jobs:
  cascade-next:
    if: github.event.pull_request.merged == true
    runs-on: ubuntu-latest

    steps:
      - name: Create and process next cascade PR
        uses: actions/github-script@v7
        with:
          github-token: ${{ secrets.CASCADE_GITHUB_TOKEN }}
          script: |
```

- `jobs: cascade-next:`
  Defines a single job called `cascade-next`.
- `if: github.event.pull_request.merged == true`
  Only run this job when the closed PR is actually merged, not just closed without merge.
- `runs-on: ubuntu-latest`
  Uses a GitHub-hosted Ubuntu runner.
- Step uses `actions/github-script@v7` with a dedicated PAT (`CASCADE_GITHUB_TOKEN`), and everything under `script: |` is the JavaScript that does the cascade logic.

3. Basic context (owner, repo, base branch, merged PR)

```
const owner = context.repo.owner;
const repo = context.repo.repo;

const baseBranch = context.payload.pull_request?.base?.ref;
const mergedPrNumber = context.payload.pull_request?.number;

core.info(`Event: ${context.eventName}`);
```

```
core.info(`Base branch: ${baseBranch}`);
core.info(`Merged PR number: ${mergedPrNumber}`);
```

Example: PR #42 merged into `release/app_1.02.00` in `my-org/payments-service`.

- `owner` = `"my-org"`
- `repo` = `"payments-service"`
- `baseBranch` = `"release/app_1.02.00"`
- `mergedPrNumber` = `42`

Logged to the Actions output for debugging.

4. Guard: baseBranch must exist

```
if (!baseBranch) {
  core.setFailed('Base branch could not be determined.');
  return;
}
```

If something is wrong with the event payload and there is no base branch, the script fails with a clear message and stops.

5. Release branch regex and `parseBranch`

```
const releaseRegex = /^release\/(?<app>[A-Za-z0-9.-]+)_(?<version>\d{1,3}\.?\d{0,2}\

function parseBranch(branch) {
  const match = branch.match(releaseRegex);
  if (!match) return null;

  const { app, version } = match.groups;

  const parts = version.split('.'); // e.g. ["1"], ["1","2"], ["1","2","03"]
```

- `releaseRegex` matches branch names like:
  - `release/hnw-services-2_1.02.03`
  - `release/hnw-services-2_1.2`
  - `release/app_2`
    and optionally with suffixes, e.g. `release/app_1.02.03-hotfix`.

  It extracts:
  - `app`: e.g. `hnw-services-2`
  - `version`: e.g. `1.02.03`
- `parseBranch(branch)`:

- Calls `branch.match(releaseRegex)`.
- If it does not match (like `main`, `develop`), returns `null`.
- Else pulls out `{ app, version }` and splits `version` by `.` into `parts`, such as:
  - `"1"` → `["1"]`
  - `"1.2"` → `["1","2"]`
  - `"1.02.03"` → `["1","02","03"]`

Next inside `parseBranch`:

```
  function toMinorOrPatchValue(str) {
    if (str === undefined || str.length === 0) {
      return 0; // missing -> 0
    }
    if (str.length === 2) {
      // 2 digits: plain number, "02" -> 2, "99" -> 99
      return parseInt(str, 10);
    }
    // 1 digit: tens, "2" -> 20, "3" -> 30, etc.
    const digit = parseInt(str, 10);
    return digit * 10;
  }

  const major = parseInt(parts[^0], 10);
  const minor = toMinorOrPatchValue(parts[^1]);
  const patch = toMinorOrPatchValue(parts[^2]);

  const versionParts = [major, minor, patch];

  return {
    branch,
    app,
    version,
    versionParts
  };
}
```

- `toMinorOrPatchValue(str)`:
  - Missing (`undefined` or empty) → 0.
  - Two digits (e.g. `"02"`, `"10"`) → parsed directly: `2`, `10`.
  - One digit (e.g. `"2"`) → tens: `"2"` → 20, `"3"` → 30.
- `major = parseInt(parts[^0], 10);`
- `minor = toMinorOrPatchValue(parts[^1]);`
- `patch = toMinorOrPatchValue(parts[^2]);`
- Example conversions:
  - `"1.02.03"` → `[major, minor, patch] = [1, 2, 3]`
  - `"1.2.03"` → `[1, 20, 3]`

    - `"1"` → `[1, 0, 0]`
- `versionParts = [major, minor, patch]`
    - Used for numeric sorting.
- Finally returns an object:

```
{
  branch: "release/hnw-services-2_1.02.03",
  app: "hnw-services-2",
  version: "1.02.03",
  versionParts: [1, 2, 3]
}
```

If the name doesn't match the release pattern, `parseBranch` returns `null`.

6. `compareVersionOnly`: numeric sorting of releases

```
function compareVersionOnly(a, b) {
  const len = Math.max(a.versionParts.length, b.versionParts.length);
  for (let i = 0; i < len; i++) {
    const av = a.versionParts[i] ?? 0;
    const bv = b.versionParts[i] ?? 0;
    if (av < bv) return -1;
    if (av > bv) return 1;
  }
  return 0;
}
```

- Takes two parsed branch objects `a` and `b`.
- Compares them element-wise by `versionParts`:
    - If a's major < b's major → a comes first.
    - If majors equal, compares minor, then patch.
- Example:
    - `a.versionParts = [1, 2, 3]` (1.02.03)
    - `b.versionParts = [1, 20, 3]` (1.2.03)
      → at index 1: 2 < 20 → `compareVersionOnly(a, b)` returns `-1`, so 1.02.03 < 1.2.03 in the sorted list.

Used by `.sort(compareVersionOnly)` later.

7. Helpers: sleep, getPullRequest, pollMergeability

```
async function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

- `sleep(ms)` returns a Promise that resolves after `ms` milliseconds.
- Example: `await sleep(2000)` pauses for 2 seconds.

```
async function getPullRequest(prNumber) {
  const response = await github.rest.pulls.get({
    owner,
    repo,
    pull_number: prNumber
  });
  return response.data;
}
```

- Uses GitHub API to fetch PR details by number, returning the PR object.

```
async function pollMergeability(prNumber, maxAttempts = 10, delayMs = 2000) {
  let pr = null;

  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    pr = await getPullRequest(prNumber);

    core.info(
      `Poll ${attempt}/${maxAttempts} for PR #${prNumber}: mergeable=${pr.mergeabl
    );

    if (pr.mergeable !== null) {
      return pr;
    }

    if (attempt < maxAttempts) {
      await sleep(delayMs);
    }
  }

  return pr;
}
```

- Polls GitHub for PR mergeability:
  - Up to `maxAttempts` times (default 10).
  - Waits `delayMs` ms between attempts (default 2000 ms).
- On each attempt:
  - Logs the values `mergeable` and `mergeable_state`.[2]
  - If `mergeable` becomes not `null` (true or false), returns the PR.
- If after all attempts it's still `null`, returns the last PR state.

8. Helpers: addLabel, addComment, mergePullRequest

```
async function addLabel(prNumber, label) {
  try {
    await github.rest.issues.addLabels({
      owner,
      repo,
      issue_number: prNumber,
      labels: [label]
    });
  } catch (error) {
    core.warning(`Could not add label '${label}' to PR #${prNumber}. ${error.messag
  }
}
```

- Adds a label to PR `prNumber`.
- Uses issues API (`addLabels`).
- On failure, logs a warning but does not fail the whole job.

```
async function addComment(prNumber, body) {
  try {
    await github.rest.issues.createComment({
      owner,
      repo,
      issue_number: prNumber,
      body
    });
  } catch (error) {
    core.warning(`Could not add comment to PR #${prNumber}. ${error.message}`);
  }
}
```

- Adds a comment to PR `prNumber` with given `body`.
- Same error-handling pattern as `addLabel`.

```
async function mergePullRequest(prNumber) {
  await github.rest.pulls.merge({
    owner,
    repo,
    pull_number: prNumber,
    merge_method: 'merge'
  });
}
```

- Merges PR `prNumber` via the GitHub API, using the standard `merge` method.

9. Parse the current base release branch

```
const current = parseBranch(baseBranch);
if (!current) {
  core.info(`Base branch '${baseBranch}' does not match the release cascade pattern.
  return;
}
```

- `current` is the parsed representation of `baseBranch`:
  - If `baseBranch = "release/app_1.02.00"`, `current` is an object with `app`, `version`, `versionParts`, etc.
- If `baseBranch` does **not** match `releaseRegex` (e.g. it's `develop`, `main`), `parseBranch` returns `null`, so:
  - Logs that the base branch doesn't match the release cascade pattern.
  - Returns early, doing no cascade.

This ensures only merges into `release/*` branches trigger cascade logic.

10. List branches and build `eligibleBranches`

```
const branches = await github.paginate(github.rest.repos.listBranches, {
  owner,
  repo,
  per_page: 100
});

const eligibleBranches = branches
  .map(branch => parseBranch(branch.name))
  .filter(Boolean)
  .filter(branch => branch.app === current.app)
  .sort(compareVersionOnly);
```

- `branches`:
  - Uses `github.paginate` on `repos.listBranches` to fetch **all branches** in the repo (in pages of 100).[3]
  - Result: an array of branch objects, each with `name` like `"main"`, `"develop"`, `"release/app_1.02.00"`, etc.
- `branches.map(branch => parseBranch(branch.name))`:
  - Converts each branch name to parsed form or `null` if it doesn't match the release pattern.
- `.filter(Boolean)`:
  - Filters out `null` entries, leaving only the release branches matching `releaseRegex`.
- `.filter(branch => branch.app === current.app)`:
  - Keeps only branches with the same `app` as the current base branch.

- Example:
  - `current.app = "hnw-services-2"`
  - Only keep branches like `release/hnw-services-2_1.00.00`, `release/hnw-services-2_1.02.03`, etc.
  - Drop release branches for other apps.

- `.sort(compareVersionOnly)`:
  - Sorts these branches by numeric version (using `versionParts` comparison).
  - Example result:

```
eligibleBranches = [
  { branch: "release/hnw-services-2_1.00.00", ... },
  { branch: "release/hnw-services-2_1.02.03", ... },
  { branch: "release/hnw-services-2_1.2.03",  ... }
];
```

11. Check for no eligible branches, log ordered list

```
if (eligibleBranches.length === 0) {
  core.info(`No eligible release branches found for app '${current.app}'. Exiting.`
  return;
}

core.info(
  `Eligible ordered branches for app '${current.app}': ${eligibleBranches.map(branc
);
```

- If there are no release branches for this `app` (should be rare), it logs and exits.
- Otherwise, logs something like:

> Eligible ordered branches for app 'hnw-services-2': release/hnw-services-2_1.00.00 -> release/hnw-services-2_1.02.03 -> release/hnw-services-2_1.2.03

This is very useful in the Actions log for debugging.

12. Find current index and basic next-branch variables

```
const currentIndex = eligibleBranches.findIndex(branch => branch.branch === current.
if (currentIndex === -1) {
  core.setFailed(`Current branch '${current.branch}' was not found in the eligible b
  return;
}

let nextBranch;
let isFinalMergeToDevelop = false;
```

- `currentIndex`:

- Finds where in `eligibleBranches` the `current.branch` sits.
- Example:
  - If `current.branch = "release/hnw-services-2_1.02.03"`, and ordered list is `[1.00.00, 1.02.03, 1.2.03]`:
    - `currentIndex = 1`.

- If `currentIndex === -1`:
  - Something is inconsistent (current branch not in list).
  - Fail with `core.setFailed` and exit.
- `nextBranch` and `isFinalMergeToDevelop` are declared:
  - `nextBranch` will later be either the next release branch or `"develop"`.
  - `isFinalMergeToDevelop` indicates whether this is the final step into `develop`.

13. Decide `nextBranch` and `isFinalMergeToDevelop`

```
if (currentIndex === eligibleBranches.length - 1) {
  nextBranch = 'develop';
  isFinalMergeToDevelop = true;
  core.info(`Branch '${current.branch}' is the last eligible release branch. Final P
} else {
  nextBranch = eligibleBranches[currentIndex + 1].branch;
  core.info(`Next cascade target branch: ${nextBranch}`);
}
```

- If `currentIndex` is the last index in the `eligibleBranches` array:
  - Current release is the highest version for this app.
  - So:
    - `nextBranch = "develop"`
    - `isFinalMergeToDevelop = true`
    - Example:
      - `current = "release/app_2.00.00"` is last → next target is `develop`.
- Else (not last):
  - Take the next release branch in the version list:
    - `nextBranch = eligibleBranches[currentIndex + 1].branch`
  - Example:
    - `current = "release/app_1.02.00"` (index 1) → next is `"release/app_2.00.00"` (index 2).

This is the core of the cascade chain.

14. List open PRs into `nextBranch` and detect existing cascade PR

```
const openPullRequests = await github.paginate(github.rest.pulls.list, {
  owner,
  repo,
  state: 'open',
  base: nextBranch,
  per_page: 100
});

const existingPr = openPullRequests.find(pr => pr.head.ref === current.branch);
if (existingPr) {
  core.info(`An open cascade PR already exists: #${existingPr.number} (${current.br
  return;
}
```

- `openPullRequests`:
  - Fetches all **open PRs** whose base (`target`) is `nextBranch`.
  - Example:
    - `nextBranch = "release/app_2.00.00"`
    - Might return:
      - PR #51: head = `"release/app_1.02.00"`, base = `"release/app_2.00.00"`
      - PR #52: head = `"hotfix/bug"`, base = `"release/app_2.00.00"`
- `existingPr`:
  - Finds a PR where `head.ref === current.branch`.
  - If `current.branch = "release/app_1.02.00"` and PR #51 already exists (`head.ref = "release/app_1.02.00"`), `existingPr` becomes that PR.
- If such a PR is found:
  - Logs:
    - `An open cascade PR already exists: #51 (release/app_1.02.00 -> release/app_2.00.00).`
  - Returns early, so you don't create duplicate cascade PRs on re-runs.

This is a natural place to stop the first third. The next response will start from:

```
const title = isFinalMergeToDevelop
  ? `[Cascade Final] ${current.branch} -> ${nextBranch}`
  : `[Cascade] ${current.branch} -> ${nextBranch}`;
```

and go through PR creation, mergeability handling, and final states.

Before moving on: is everything up to and including `existingPr` aligned with your understanding, or do you want to tweak any of the behaviors (like when we stop vs when we proceed) before we document the rest?

⁂

1. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/managing-a-branch-protection-rule
2. https://docs.github.com/en/enterprise-server@3.14/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets
3. https://docs.github.com/en/rest/branches
4. cascade-next-pr.yml